

Mobile React Native T1 - M1S6

Mini-Projeto Avaliativo - Módulo 01 - Semana 06

Este documento descreve o Mini-Projeto Avaliativo da turma de Programação Desenvolvimento Mobile React Native Turma 01.

SUMÁRIO

1. CONTEXTUALIZAÇÃO	1
2. DESAFIO	2
3. RESULTADOS ESPERADOS (ENTREGA)	3
4. REQUISITOS DAS TAREFAS	3
4.1. Organização do projeto	3
4.2. Requisitos funcionais	4
RF01 – Criar o perfil da pessoa usuária via terminal (interativo)	4
RF02 – Criar um catálogo de conteúdos	5
RF03 – Calcular compatibilidade com cada conteúdo	5
RF04 – Classificar a compatibilidade	6
RF05 – Listar habilidades faltantes	6
RF06 – Encontrar a vaga com maior compatibilidade	6
RF07 – Gerar uma recomendação personalizada	6
RF08 – Usar métodos de array	7
RF09 – Criar uma classe	7
RF10 – Usar herança	7
RF11 – Demonstrar uso do this	8
RF12 – Usar callback	8
RF13 – Usar closure	8
RF14 – Usar Promise e async/await	9
RF15 – Criar um menu interativo com opções	9
4.3. REQUISITOS TÉCNICOS	10
4.4. ORGANIZAÇÃO DO KANBAN	12
4.5. VERSIONAMENTO COM GITHUB	13
4.6. GRAVAÇÃO DE VÍDEO	13
5. CRITÉRIOS DE AVALIAÇÃO	14
6. CHECKLIST FINAL DE ENTREGA	16
7. MODELO DE README.MD (SUGESTÃO)	17

1. CONTEXTUALIZAÇÃO

O mercado de desenvolvimento front-end exige que você saiba demonstrar raciocínio lógico, organização, capacidade de resolver problemas, domínio inicial de JavaScript e boa comunicação técnica. Mesmo antes de criar interfaces com HTML, CSS, DOM ou React, já é possível construir um projeto útil para o seu portfólio usando apenas JavaScript.

Plataformas de streaming de filmes, séries e música que você já usa no dia a dia funcionam, por trás das telas, com motores de recomendação: sistemas que comparam o perfil de cada pessoa usuária com um catálogo de conteúdos para sugerir o que assistir a seguir. Neste mini-projeto, você vai construir uma versão simplificada — e totalmente interativa — desse tipo de sistema.

Diferente de exercícios anteriores rodados direto no console do navegador, este projeto roda via terminal (Node.js), com um fluxo real de perguntas e respostas: a própria pessoa usuária digita seu nome, sua idade e seus gêneros favoritos, e o sistema reage a cada resposta, calculando compatibilidades em tempo real. É a mesma lógica por trás do onboarding de qualquer app de streaming quando você cria uma conta pela primeira vez.

Esse mini-projeto trabalha diretamente os conteúdos já estudados: lógica de programação; tipos de dados; condicionais; operadores; escopo; laços de repetição; funções; arrow functions; arrays; métodos de array; objetos; classes; construtores; herança; uso do this; callbacks; closures; Promises; async/await; entrada de dados via terminal com prompt-sync; GitHub; GitHub Desktop; Kanban.

O projeto também reforça sua organização profissional usando o Kanban para visualizar tarefas em colunas e acompanhar o progresso do trabalho em andamento. Sendo uma maneira de visualizar seu trabalho, analisar os itens em andamento e melhorar o fluxo de evolução do seu código. Por exemplo, usando o GitHub Flow com branches descritivas, commits com mensagens claras e uso de branches para trabalhar sem afetar a branch principal.

2. DESAFIO

Você acaba de ser contratado(a) como desenvolvedor(a) júnior pela PlayNow, uma startup de streaming em crescimento acelerado. O time de Produto descobriu que boa parte das pessoas assinantes cancela o serviço na primeira semana porque “não encontra nada bom para assistir” logo no início. Você foi chamado(a) para construir o protótipo de um motor de recomendação (engine) que já converse com a pessoa usuária no primeiro acesso, pelo terminal, e sugira o que assistir.

Você deverá desenvolver um mini-projeto em JavaScript (Node.js) chamado:

CineMatch JS: Onboarding Interativo de Recomendação de Streaming

O objetivo do projeto é criar um sistema interativo que, a partir das respostas digitadas pela pessoa usuária no terminal, compare seu perfil com um catálogo de filmes e séries fictícios.

A aplicação deverá analisar:

- quais gêneros a pessoa usuária mais gosta (perguntado diretamente pelo terminal);
- quais gêneros cada conteúdo do catálogo possui;
- quais gêneros ainda não foram explorados por ela;
- qual é o percentual de compatibilidade com cada conteúdo;
- qual conteúdo possui maior compatibilidade;
- qual recomendação personalizada deve ser dada à pessoa usuária;
- e permitir, por meio de um menu interativo, que ela escolha sua próxima ação dentro do programa

Maneira de execução

Poderá executar o código de uma destas formas:

- via terminal, rodando node cinematch.js (forma recomendada);
- usando o terminal integrado do VS Code;
- usando um ambiente online que suporte Node.js

Para capturar as respostas digitadas no terminal, será usada a biblioteca prompt-sync. Ela permite pausar a execução do script e esperar a pessoa usuária digitar uma resposta antes de continuar — algo que o console do navegador não faz de forma nativa e síncrona.

O Node.js executa JavaScript fora do navegador, direto no seu computador, o que permite ler e escrever no terminal. Já o VS Code possui um terminal integrado e suporte oficial a depuração em JavaScript, mas isso não deve ser cobrado como dependência obrigatória do projeto.

3. RESULTADOS ESPERADOS (ENTREGA)

Ao final do mini-projeto, o estudante ou squad (pequeno grupo de até 03 pessoas) deverá entregar:

- Um repositório público no GitHub com o projeto.
- Um arquivo JavaScript principal, preferencialmente chamado cinematch.js..
- Um arquivo README.md explicando o projeto.
- Um quadro Kanban com as tarefas realizadas.
- Um histórico de commits no GitHub.
- Um vídeo de até 5 minutos demonstrando o funcionamento.
- Os links submetidos no AVA.

Entrega no AVA

- A entrega **deverá ser submetida na tarefa do AVA:** "Módulo 01 - Mini-Projeto Avaliativo"
- **Prazo:** 10/08/2026 até às 22h
- **Peso do mini-projeto na nota:** Avaliação M1.1 – 25% da nota do módulo

Links obrigatórios

O estudante deverá enviar no AVA:

- link do repositório público no GitHub;
- link do quadro Kanban utilizado no projeto.
- link do vídeo demonstrando o funcionamento.

4. REQUISITOS DAS TAREFAS

4.1. Organização do projeto

A estrutura mínima do projeto deverá ser:

```
cinematch-js/  
├── cinematch.js  
├── package.json  
└── README.md
```

Para squads (grupos de até ~03 alunos) que quiserem organizar melhor, será aceita a seguinte estrutura:

```
cinematch-js/  
├── README.md  
├── cinematch.js  
├── package.json  
└── planejamento/  
    └── tarefas-kanban.md
```

Não é necessário criar várias pastas. O foco é demonstrar domínio dos conceitos de JavaScript estudados até a Semana 06.

4.2. Requisitos funcionais

RF01 – Criar o perfil da pessoa usuária via terminal (interativo)

O projeto deverá capturar os dados da pessoa usuária digitados no terminal, usando a biblioteca prompt-sync. Exemplo:

```
const prompt = require('prompt-sync')({ sigint: true });  
  
const nome = prompt('Qual é o seu nome? ');  
const idade = Number(prompt('Qual é a sua idade? '));  
const generosInput = prompt(  
  'Quais gêneros você mais gosta? (separe por vírgula, ex: Ação, Comédia, Terror): '  
);  
  
const usuario = {  
  nome: nome,  
  idade: idade,  
  generosFavoritos: generosInput.split(',').map((g) => g.trim()),  
};
```

Lembrando, no fim será um Objeto Simples (usuario)

A pessoa usuária poderá digitar qualquer nome e qualquer combinação de gêneros — o sistema deve funcionar corretamente com respostas diferentes, e não apenas com o exemplo acima.

RF02 – Criar um catálogo de conteúdos

O projeto deverá ter um array com pelo menos 03 conteúdos fictícios (filmes ou séries).
Exemplo:

```
const catalogo = [  
  {  
    id: 1,  
    titulo: "Fronteira Digital",  
    tipo: "Série",  
    generos: ["Ação", "Ficção Científica"],  
    duracaoMinutos: 45,  
    temporadas: 2  
  },  
  {  
    id: 2,  
    titulo: "Risadas de Sábado",  
    tipo: "Filme",  
    generos: ["Comédia", "Romance"],  
    duracaoMinutos: 98  
  },  
  {  
    id: 3,  
    titulo: "Sombras do Porão",  
    tipo: "Filme",  
    generos: ["Terror", "Suspense"],  
    duracaoMinutos: 110  
  }  
];
```

RF03 – Calcular compatibilidade com cada conteúdo

O sistema deverá comparar os gêneros favoritos da pessoa usuária com os gêneros de cada conteúdo. Exemplo de regra:

```
compatibilidade = quantidade de gêneros em comum / total de gêneros do conteúdo *  
100
```

Exemplo de resultado esperado no console:

```
Título: Fronteira Digital  
Tipo: Série  
Compatibilidade: 50%  
Gêneros em comum: Ação  
Gêneros não explorados: Ficção Científica  
Classificação: Média afinidade
```

RF04 – Classificar a compatibilidade

O projeto deverá classificar cada vaga conforme o percentual calculado.

Percentual	Classificação
80% a 100%	Alta afinidade
50% a 79%	Média afinidade
0% a 49%	Baixa afinidade

O estudante deverá usar pelo menos uma estrutura de decisão:

- if-else;
 - switch-case;
 - ou operador ternário.
-

RF05 – Listar habilidades faltantes

O sistema deverá mostrar quais gêneros a pessoa usuária ainda não indicou gostar, para cada conteúdo. Exemplo:

Para "Sombras do Porão", você ainda não explorou:
- Suspense

RF06 – Encontrar a vaga com maior compatibilidade

O sistema deverá identificar qual conteúdo do catálogo possui maior aderência ao perfil da pessoa usuária. Exemplo:

Recomendação principal:
Fronteira Digital (Série)
Compatibilidade: 100%

Pode ser usado reduce, for, for...of ou outra estratégia vista em aula.

RF07 – Gerar uma recomendação personalizada

Com base nos gêneros não explorados, o sistema deverá sugerir o próximo gênero a testar. Exemplo:

Recomendação personalizada para Carlos:

Você já curte Ação — que tal arriscar um pouco de Ficção Científica?
"Fronteira Digital" pode ser um ótimo próximo título.

RF08 – Usar métodos de array

O projeto deverá usar pelo menos 3 métodos de array entre: map; filter; find; every; reduce.

Exemplo de usos possíveis:

Método	Uso sugerido
map	gerar uma nova lista com os resultados de compatibilidade
filter	separar os conteúdos com afinidade alta
find	buscar um conteúdo específico pelo título
every	verificar se a pessoa usuária curte todos os gêneros de um conteúdo
reduce	calcular o conteúdo com maior compatibilidade ou o tempo total do catálogo

RF09 – Criar uma classe

O estudante deverá criar pelo menos uma classe simples. Exemplo:

```
class Conteudo {  
  constructor(titulo, tipo, generos, duracaoMinutos) {  
    this.titulo = titulo;  
    this.tipo = tipo;  
    this.generos = generos;  
    this.duracaoMinutos = duracaoMinutos;  
  }  
  
  exibirResumo() {  
    return `${this.titulo} (${this.tipo}) — ${this.duracaoMinutos}  
min`;  
  }  
}
```

RF10 – Usar herança

O estudante deverá criar uma classe que herda de outra. Exemplo:

```
class Serie extends Conteudo {
  constructor(titulo, generos, duracaoMinutos, temporadas) {
    super(titulo, "Série", generos, duracaoMinutos);
    this.temporadas = temporadas;
  }

  exibirTemporadas() {
    return `${this.titulo} tem ${this.temporadas} temporada(s)`;
  }
}
```

RF11 – Demonstrar uso do this

O projeto deverá ter pelo menos um método que utilize this. Exemplo:

```
exibirResumo() {
  return `${this.titulo} (${this.tipo}) - ${this.duracaoMinutos} min`;
}
```

RF12 – Usar callback

Criar uma função que receba outra função como parâmetro. Exemplo:

```
function finalizarOnboarding(nomeUsuario, callback) {
  console.log("Onboarding finalizado.");
  callback(nomeUsuario);
}

function exibirMensagemFinal(nome) {
  console.log(`${nome}, aproveite sua maratona! Bom streaming.`);
}
```

RF13 – Usar closure

Criar uma função que mantenha um valor interno. Exemplo:

```
function criarContadorDeRecomendacoes() {
  let total = 0;

  return function () {
    total++;
    return total;
  };
}
```

RF14 – Usar Promise e async/await

Simular o carregamento do catálogo como se os dados viessem de um servidor de streaming. Não é necessário usar API real. Exemplo:

```
function buscarCatalogoSimulado() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(catalogo);
    }, 1000);
  });
}

async function iniciarSistema() {
  const catalogoCarregado = await
  buscarCatalogoSimulado();
  console.log("Catálogo carregado com sucesso!");
  console.log(catalogoCarregado);
}
```

RF15 – Criar um menu interativo com opções

Depois do onboarding (perfil + catálogo carregado), o sistema deverá exibir um menu de opções no terminal, repetindo a pergunta até a pessoa usuária escolher sair. Exemplo de menu:

```
===== CineMatch JS =====
1 - Ver meu perfil
2 - Ver catálogo completo
3 - Calcular compatibilidade com todos os conteúdos
4 - Ver o conteúdo mais recomendado
5 - Sair
```

O menu deverá usar um laço de repetição (recomendado: do-while, pois o menu precisa ser exibido pelo menos uma vez) combinado com switch-case (ou if-else) para decidir o que fazer a cada opção escolhida. Exemplo:

```
let opcao;

do {
  console.log("\n===== CineMatch JS =====");
  console.log("1 - Ver meu perfil");
  console.log("2 - Ver catálogo completo");
  console.log("3 - Calcular compatibilidade com todos os conteúdos");
  console.log("4 - Ver o conteúdo mais recomendado");
```

```

console.log("5 - Sair");

opcao = prompt("Escolha uma opção: ");

switch (opcao) {
  case "1":
    exibirPerfil(usuario);
    break;
  case "2":
    exibirCatalogo(catalogo);
    break;
  case "3":
    calcularCompatibilidades(usuario, catalogo);
    break;
  case "4":
    exibirRecomendacaoPrincipal(usuario, catalogo);
    break;
  case "5":
    console.log("Até a próxima maratona!");
    break;
  default:
    console.log("Opção inválida, tente novamente.");
}
} while (opcao !== "5");

```

Cada opção do menu deverá chamar funções já criadas para os outros requisitos (RF03 a RF07) — o menu é só a porta de entrada, não é necessário duplicar lógica.

4.3. REQUISITOS TÉCNICOS

Conteúdo	Obrigatório?	Como demonstrar
Kanban	Sim	Quadro com tarefas do projeto
Node.JS	Sim	Necessário para executar o projeto e usar prompt-sync
VS Code	Sim	Usar como editor, se possível
Extensões necessárias	Sim	Citar no README as extensões usadas ou recomendadas
Como a internet funciona	Sim	Explicação curta no README
Arquitetura cliente-servidor	Sim	Explicação curta + Promise simulando busca
Versionamento	Sim	Commits no GitHub
GitHub	Sim	Repositório público

Conteúdo	Obrigatório?	Como demonstrar
GitHub Desktop	Sim	Pode ser usado para commits e push
GitFlow simplificado	Sim	Branches simples
Lógica de programação	Sim	Cálculo de compatibilidade
JavaScript	Sim	Arquivo .js
Tipos de dados	Sim	Strings, números, arrays, objetos e booleanos
If-else	Sim	Classificação de compatibilidade
Switch-case	Sim	Usado no menu interativo de opções (RF15); pode também ser usado na classificação
Operador ternário	Opcional	Pode ser usado em mensagens
Operadores lógicos e matemáticos	Sim	Percentuais e comparações
Var, let e const	Sim	Priorizar const e let; explicar var no README
For, while ou do-while	Sim	Listagem, análise do catálogo e loop do menu interativo (recomendado: do-while)
Funções	Sim	Separação das regras
Arrow functions	Sim	Métodos de array
Arrays	Sim	Lista de conteúdos e gêneros
Map, every, find, filter e reduce	Sim	Usar pelo menos 3
Objetos simples	Sim	Candidato e vagas
Chaves e valores	Sim	Manipulação dos objetos
POO	Sim	Classe Vaga
Classes	Sim	Classe principal
Objetos	Sim	Instâncias ou objetos literais
Atributos	Sim	Empresa, cargo, requisitos etc.
Métodos	Sim	Exibir resumo ou calcular algo

Conteúdo	Obrigatório?	Como demonstrar
Construtores	Sim	constructor
Herança	Sim	Serie extends Conteudo
Uso do this	Sim	Dentro de método da classe
Closures	Sim	Contador de análises
Promises	Sim	Simulação de busca do catálogo
Async/Await	Sim	Função iniciarSistema
Callbacks	Sim	Mensagem final ou processamento

4.4. ORGANIZAÇÃO DO KANBAN

- O estudante ou squad deverá organizar o trabalho em um quadro Kanban. Colunas obrigatórias: Backlog; A Fazer; Em Andamento; Concluído;
- Pode usar: Trello; GitHub Projects; Notion; quadro simples no README; planilha; imagem do quadro:
- **Tarefas sugeridas para o Kanban:**
 - Criar repositório no GitHub
 - Criar arquivo cinematch.js
 - Configurar Node.js e instalar prompt-sync (npm install prompt-sync)
 - Criar README.md
 - Criar coleta interativa do perfil (Objeto: nome, idade, gêneros) via prompt-sync
 - Criar array de vagas
 - Criar função para calcular compatibilidade
 - Criar função para listar habilidades faltantes
 - Criar classificação por percentual
 - Usar map/filter/find/reduce/every
 - Criar classe Conteudo
 - Criar classe Serie
 - Aplicar herança
 - Usar this em método de classe
 - Criar função com callback
 - Criar função com closure
 - Criar Promise simulada
 - Criar função async/await
 - Criar menu interativo com switch-case e loop do-while
 - Testar toda a interação no terminal, do início ao fim
 - Atualizar README
 - Enviar links no AVA

4.5. VERSIONAMENTO COM GITHUB

O projeto deverá ter um repositório público no GitHub.

Branches mínimas:

- Para projeto individual:
 - main;
 - develop;
 - feat/perfil-interativo;
 - docs/readme.
- Para squad (até 03 pessoas):
 - main
 - develop
 - feat/perfil-catalogo
 - feat/calculo-compatibilidade
 - feat/classes-poo
 - docs/readme-video

Commits mínimos:

Mini-Projeto individual:

- mínimo de 5 commits

Mini-Projeto em squad:

- mínimo de 8 commits

Exemplos de commits:

- feat: cria estrutura inicial do projeto
- feat: adiciona coleta interativa do perfil com prompt-sync
- feat: adiciona catálogo de conteúdos
- feat: implementa cálculo de compatibilidade
- feat: adiciona classe Serie com herança
- feat: cria simulação de busca com promise e async await
- docs: atualiza readme com instruções
- fix: corrige cálculo de percentual

O GitHub Flow recomenda criar branches com nomes curtos e descritivos, trabalhar sem afetar a branch principal e fazer commits com mensagens descritivas.

4.6. GRAVAÇÃO DE VÍDEO

Além do desenvolvimento das tarefas você deverá gravar um vídeo (ou seu grupo de até 03 alunos, um vídeo por grupo), com tempo **máximo de 5 minutos**, abordando os seguintes questionamentos:

- Qual o objetivo do sistema? E demonstração de funcionamento.

- O que deve ser realizado para executar o sistema?
- Como você organizou as tarefas antes de começar a desenvolver?
- Quais as *branches* você criou e quais os objetivos para cada uma?
- Você acha que faltou algo no seu código que você poderia melhorar?

Você poderá gravar na vertical ou na horizontal. É importante que apareça seu rosto e esteja em um local com boa iluminação e que seja possível ouvir sua voz bem. Para realizar a entrega do vídeo, coloque em uma pasta do Google Drive em modo leitor para qualquer pessoa com o link (ou subo via YouTube como vídeo não listado), e compartilhe o mesmo na submissão do projeto no AVA. Uma dica interessante é você inserir o vídeo no `readme.md` do seu projeto no repositório do GitHub. É um requisito que qualquer pessoa consiga visualizar seu vídeo via algum link.

5. CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto.

O mesmo possui variação de nota de 0 (zero) a 10 (dez) como nota mínima e máxima, e possui peso de 25% sobre a avaliação do módulo 01 do curso.

Serão desconsiderados e atribuída a nota 0 (zero) os projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas.

Lembre-se: Você está livre para utilizar outras soluções como base, mas não é permitida a cópia. Ou seja, você está livre para consultar materiais, documentação, exemplos e ferramentas de apoio, desde que consiga adaptar a solução ao desafio proposto e explicar o código entregue com segurança e confiança.

Uso do GitHub e Readme.md				
Nº	Critério de Avaliação	0	0,75	1,50
01	Versionamento com branches e commits	O repositório do projeto não apresenta branches e commits.	O repositório do projeto apresenta parte das branches e commits distintos e nomeadas padronizadamente para cada funcionalidade desenvolvida.	O repositório do projeto apresenta branches e commits distintos e nomeadas padronizadamente para cada funcionalidade desenvolvida.
Nº	Critério de Avaliação	0	0,25	0,50
02	Organização dos arquivos no repositório	O repositório do projeto não apresenta os arquivos estruturados conforme as instruções.	O repositório do projeto apresenta parte dos arquivos estruturados conforme as instruções.	O repositório do projeto apresenta os arquivos estruturados conforme as instruções com <code>Readme.md</code> estruturado

Apresentação do Projeto

Nº	Critério de Avaliação	0,00	1,00	2,00
03	Gravação de vídeo	Não gravou o vídeo de apresentação.	Gravou o vídeo de apresentação, mas não apresentou todos os requisitos do item 4.6.	O vídeo apresentou todos os requisitos do item 4.6.
Nº	Critério de Avaliação	0,00	0,25	0,50
04	Quadro Kanban	O quadro Kanban não foi configurado ou está em branco.	O quadro foi criado, mas faltam colunas obrigatórias ou as tarefas não foram movimentadas/atualizadas corretamente.	O quadro Kanban está estruturado com as colunas obrigatórias (<i>Backlog, A Fazer, Em Andamento, Concluído</i>) e as tarefas foram distribuídas de forma coerente com o desenvolvimento do projeto.

Desenvolvimento do Projeto

Nº	Critério de Avaliação	0,00	0,40	0,80
05	Interação inicial do usuário	O software não realizou a coleta de dados do perfil do usuário via terminal, ou a implementação não utilizou a biblioteca <code>prompt-sync</code> , comprometendo o funcionamento da aplicação.	O software realizou parcialmente a coleta de dados do perfil do usuário via terminal utilizando a biblioteca <code>prompt-sync</code> , mas apresentou falhas na estruturação e/ou armazenamento das informações coletadas ou do catálogo de dados por meio de objetos e arrays.	O software realizou a coleta de dados do perfil do usuário via terminal utilizando a biblioteca <code>prompt-sync</code> , estruturando e armazenando as informações coletadas, bem como o catálogo de dados, por meio de objetos e arrays.
06	Cálculo de compatibilidade e gêneros	O código não realizou o cálculo de compatibilidade de perfis, não mapeou os gêneros não explorados e/ou não aplicou a classificação de dados conforme as regras de negócio definidas no projeto.	O código apresentou falhas no cálculo de compatibilidade, no mapeamento de gêneros ou na classificação.	O código realizou o cálculo de compatibilidade de perfis, mapeou os gêneros não explorados e aplicou a classificação de dados de acordo com as regras de negócio definidas no projeto.
07	Uso de funções, arrow functions, escopo, operadores, decisões e repetições	O código não apresentou funções, arrow functions, escopo, operadores, decisões e repetições.	O código-fonte apresentou modularização parcial com funções, arrow functions, estruturas de decisão ou repetição,	O código-fonte da aplicação apresentou modularização por meio de funções e <i>arrow functions</i> , manipulação de escopos e operadores, além de estruturas de

			sustentando de forma incompleta o menu interativo.	decisão e repetição que sustentam o funcionamento pleno de um menu interativo.
08	Herança de classes	O projeto não utiliza herança ou a implementação quebra a execução do software.	O projeto utiliza herança, porém a implementação apresenta inconsistências, como atributos ou métodos específicos da subclasse mal estruturados ou incompletos.	Criou uma classe que herda de outra, adicionando atributos específicos.
09	Utilização de funções avançadas	O código-fonte não apresentou funções assíncronas estruturadas.	O código-fonte apresentou <code>async/await</code> e/ou <code>Promises</code> com inconsistências na estruturação, causando bloqueios no fluxo principal da aplicação.	O código-fonte apresentou funções assíncronas estruturadas com <code>async/await</code> e <code>Promises</code> , solucionando requisições paralelas ou encadeadas sem o bloqueio do fluxo principal da aplicação.
Nº	Critério de Avaliação	0,00	0,75	1,50
10	Uso de métodos de array	O código fonte não apresentou a manipulação e transformação de dados.	O código fonte apresentou uso de menos de 3 métodos de array, ou erros na aplicação dos métodos utilizados.	O código fonte apresentou a manipulação e transformação de dados por meio do uso de, pelo menos, 3 métodos de array.

6. CHECKLIST FINAL DE ENTREGA

Antes de enviar no AVA, confira:

- ☐ Criei o repositório público no GitHub
- ☐ Criei o arquivo `cinematch.js`
- ☐ Criei o `README.md`
- ☐ Criei o quadro Kanban
- ☐ Criei a coleta interativa do perfil/Objeto (nome, idade, gêneros) via `prompt-sync`
- ☐ Criei pelo menos 3 conteúdos fictícios no catálogo
- ☐ Usei arrays
- ☐ Usei objetos
- ☐ Usei strings, números e booleanos
- ☐ Usei `if-else`, `switch-case` ou operador ternário
- ☐ Usei operadores lógicos e matemáticos
- ☐ Usei `let` e `const` corretamente
- ☐ Usei `for`, `while` ou `do-while`
- ☐ Criei um menu interativo com opções (`loop` + `switch-case`) para navegar entre as funcionalidades
- ☐ Usei funções
- ☐ Usei arrow functions

- [] Usei pelo menos 3 métodos de array
- [] Usei classe
- [] Usei construtor
- [] Usei herança
- [] Usei this
- [] Usei callback
- [] Usei closure
- [] Usei Promise
- [] Usei async/await
- [] Fiz commits no GitHub
- [] Testei toda a interação no terminal do início ao fim
- [] Usei branches
- [] Gravei o vídeo de até 5 minutos
- [] Coloquei o vídeo no Google Drive com permissão correta per ver vídeo via link
- [] Enviei os links no AVA até 10/08/2026 às 22h

7. MODELO DE README.MD (SUGESTÃO)

CineMatch JS

Sobre o projeto

O CineMatch JS é um simulador interativo de recomendação de streaming. O sistema conversa com a pessoa usuária pelo terminal, coleta seu perfil (nome, idade, gêneros favoritos) e compara com um catálogo de filmes e séries.

O projeto mostra:

- percentual de compatibilidade com cada conteúdo;
- gêneros em comum;
- gêneros ainda não explorados;
- conteúdo mais compatível;
- recomendação personalizada;
- menu interativo para navegar entre as funcionalidades.

Objetivo

Praticar os principais conceitos do Módulo 01:

- lógica de programação;
- JavaScript;
- tipos de dados;
- condicionais;
- operadores;
- escopo;
- laços de repetição;
- funções;
- arrow functions;
- arrays;
- métodos de array;

- objetos;
- classes;
- herança;
- this;
- callbacks;
- closures;
- Promises;
- async/await;
- entrada de dados via terminal (prompt-sync);
- GitHub;
- Kanban.

Como executar

Este projeto precisa de Node.js.

1. Clone o repositório.
2. Rode `npm install` para instalar o prompt-sync.
3. Rode `node cinematch.js`.
4. Responda as perguntas exibidas no terminal.
5. Use o menu para navegar entre perfil, catálogo, compatibilidade, recomendação e sair.

Estrutura do projeto

```
```.txt
cinematch-js/
├── cinematch.js
├── package.json
└── README.md
```.
```